I searched for the referenced files, but no file content was available to open. Therefore, I can only use the YAML content included in your message as the authoritative source.

Workflow Plan: scicomp-kanban Issue Migration

Source

- Project: scicomp-kanban
- ID: 641
- URL: https://rcode.rockefeller.edu/jbanfelder/scicomp-kanban

Destination

- Project: scicomp-kanban_cce-migration
- ID: 83486174
- URL: https://gitlab.com/rockefelleruniversity/it/infra/cce/rcode_forks/scicomp-kanban_cce_migration

---

Migration Scope

Migrate

- Issues
- Issue descriptions
- Assignees
- Labels
- Due dates
- Milestones (if present)
- Comments/notes
- Issue state (open/closed)
- Attachments referenced in issue notes (optional)

Preserve

- Original issue number
- Original author
- Original timestamps (recorded in migrated note header)
- Source project links

Exclude

- Pipelines
- Repositories

- Wikis
- Epics
- Merge Requests

Migration Architecture

```
Source GitLab (rcode)
        |
        v
Issue Collector
        |
        v
User Mapping Engine
        |
        v
 Checkpoint Store
         |
         v
 Async Migration Workers
         |
         v
 Destination GitLab
```

Repository Layout

```
gitlab-migration/

├── gitlab_migration.py
├── config.py
├── mappings.yml
├── requirements.txt
│
├── checkpoints/
│   └── migrated_issues.json
 │
├── reports/
│   ├── migration_report.json
│   └── failed_items.json
```

```
14  │
15  ├── logs/
16  │   └── migration.log
17  │
18  └── README.md
```

mappings.yml

```yaml
1  source:
2    path: "https://rcode.rockefeller.edu/jbanfelder/scicomp-
kanban"
3    project_id: 641
4    name: "scicomp-kanban"
5
6  destination:
7    path:
"https://gitlab.com/rockefelleruniversity/it/infra/cce/rcode_forks
/scicomp-kanban_cce_migration"
8    project_id: 83486174
9    name: "scicomp-kanban_cce-migration"
10
11  users:
12    jherrera:
13      dest_username: "jherrera15"
14      pat_env: "PAT_JHERRERA15"
15
16    jbanfelder:
17      dest_username: "jbanfelder"
18      pat_env: "PAT_JBANFELDER"
19
20    source_username_2:
21      dest_username: "dest2"
22      pat_env: "PAT_DEST2"
```

config.py

```python
1  import os
2
```

```
SOURCE_GITLAB_URL = os.getenv(
    "SOURCE_GITLAB_URL",
    "https://rcode.rockefeller.edu"
)

DEST_GITLAB_URL = os.getenv(
    "DEST_GITLAB_URL",
     "https://gitlab.com"
 )

 SOURCE_TOKEN = os.getenv("SOURCE_GITLAB_TOKEN")

 DEST_TOKEN = os.getenv("DEST_GITLAB_TOKEN")

 MAX_CONCURRENT_REQUESTS = int(
     os.getenv("MAX_CONCURRENT_REQUESTS", "10")
 )

 REQUEST_TIMEOUT = int(
     os.getenv("REQUEST_TIMEOUT", "60")
 )
```

README.md

```
# scicomp-kanban GitLab Issue Migration

## Overview

Migrates all issues from:

Project ID: 641

to

 Project ID: 83486174

 Features:

 - Async migration
```

````
- Dry-run mode
- Resumable checkpoints
- User remapping
- Full issue/comment migration
- Pilot migration support

---

## Install

```bash
python3 -m venv venv

source venv/bin/activate

pip install -r requirements.txt
````

Configure

Set environment variables:

```
export SOURCE_GITLAB_TOKEN=xxxxx
export DEST_GITLAB_TOKEN=xxxxx

export PAT_JHERRERA15=xxxxx
export PAT_JBANFELDER=xxxxx
```

Dry Run

```
python gitlab_migration.py \
    --dry-run \
    --limit 50
```

Pilot Migration

```
python gitlab_migration.py \
    --limit 50
```
```

Resume

```
python gitlab_migration.py \
    --resume
```

Full Migration

```
python gitlab_migration.py
```

Reports

```
reports/migration_report.json
reports/failed_items.json
logs/migration.log
```



8K1dVxMg/0asAw/HuvFnE7Hf4pNtDcga3WfvPW0jYfZ/9G2DvOu3r1730fB9vZ+2io
o5zHGXfX19SbmD7i362Hqvm4FzM2Hoq5ECdhbbiab+Ss6HIjZ54b3KWRlMw79Eg7HR
1YG9kDzzoqKivwNY0O9fhyS/XgSLsW2RMz+yDDF9emMt7ceh+BJxvgBc3iN+b4cyRP
9O2WI9s+xZtpWYHadZT/aM+IXRNrPpFyH3YFtUxeJiXY2jnfPOidS6GtGeSl+C5aHx
b2Y/3U0NGL25fE8D7haWG7z8KI9+nrXbpibPewKrRzHlsPpZgRVil8Gq+OIn81DyNb
jserRo0dZnH6tqujjPiWNh63T5lowcODAL7lAYirJZQ8sq+ptP2QdI2CbfzkgbUPsq
dah3vSzjfEEyEbvbZ3Z9Lf5vcGeuAXc21t1NofB/tjAbccvDvNufMa9ZU9sxrAn9mV
ksTcnbruXvYHaDeIPHXYwzyWoAevDGL+LV0dbmxcchkQaG1nrKGwA1uYff8rLyzczz
tORPhN5uM2IlNt05HuERuuzH2/JKsoN5GjVj3vva4I2e4jag/Un1F3B/Oz3OHvzWVM
i5uWh76boYB4SW6PvVe4aAXuK2kbZod41ksqri5RDfO/bp4yr2+Zpzga532125/ePU
usX+Qui20D3dPRyEO95izNDXON5W15IvX0uWlWIt633ZEdw5l+h7SMO3EwaLcZ+77E
/XHC7zWWCt8oi+8Ghe8B8tJHLewNQ582DvP9LXDEPDau3T7XePNHvja2jXz597A0QW
s4/b3C/J/d29YqI1dbbk/VcidkfOebZH00swBl9bP7ev4My7iWR+nV8xr4aKdsYVhx
kPzBvjvQLR/6wY5+08+Czt82XfjcRY298rx9zdusPwdT+GcH+iNbMWPZrRK9IbBixe
+vg3v2xzd503t6Sw3I9T5vlLbaxKVveBj5x/0LZi6M+n3l486NOVycIePAAbn+xNMC
W4hQOz60UPMhs1PWU7XcGXGg0v+fZhlrZMw7AC/S338E+43PqHfr+AbMnrfXfwCbZW
2sXgi3GNvTLtO/GvXvTLmAMOww2JtWhEHmW8qn0PofmcyqGk/9N8jxI2a4iyg9RsD5
hcs2mbNc/+WH/VmafffYm+l3kbUH1FxcH8mpyuU/gHvS1Tzn7S+2ltNmn7C7kvguzq
1Ud/ewvqvYXWnsg/YN793vOKN4IIxjBfv+0N4V9UtoD59aKigr/TcFYxxNjn/cHsbY
5HGT7vWwJA82iPkS7CdA+Xe32BO7td0CfGQf9DBpMQJN5IH7EfBuJ2chfHg+h/kTMr
tHUX2iFXr162e/YNn4YnvbHJ+/NR/nXcLF12D/52P8I4nrmYPthZg/i8/jd7mzqjOM
55LN5buX+OMom6L0sP3YO6/4Z3q5C1jTdCrLECZj4vgTYQWxIOd32ZXMGUz4fy4vYd
/G7Y3bfk0PT6q+a9LF/L5tHe38OxQgO5ePUzeDTcwfqBiHOxXxmfUS5L2Y5euPX8eS
eGA6H/4NyP2Juw/fHrN1sMHV7ch/G7P5s7p8h7+7YvtQVM097g1jZ3prXkm9n6nYm5
5l4++w7jdhWFzHX0LcAs5xm+yD8C6jvQ78f0a+UcaZRd3FsnSWi39GwGtGzZ097mJS
xVlvLAPq8StuNWF/aD+Dw2h85Wn1q0/YIZkzCxHt/COJ+H8r2Rg9RPhizOZkVUv4Z5
jNjLPvktLfrWuZiYrI/Ktlfkc8jzrHLZ97ew2jIkCGbWM8O9DuQ+Qykflf6fYnYsXl
5ec3sz+74vbm33w+fwDvWI+yhwbxGs5YRcDmJvr24f4aYuZiLK6Z8LGbzzaP9dvLr6
gABA7kIcN5nHCDfQSj2Tw8dSPHv0L322usDNuoh8iyMfHr+uzGmxMY3xRNyTFirW/K
uw96xSpunKzN/+xRtom4zOf+ET3gNw4YNq4zMxe8Xr87GNGOsFfaXSMa2B4j9nmjVv
tG+2A6vX5G8gvdGQwwTGHsoDzr7d0H7ve9/2hoCDpvZjzdsPqyxmX6L8PYGD9n+0Ga
f2W11D7GW1caTPva7fZtxaugcAXvzda6negVKgLex/VPGY/h5fG7avy/a/zrlAgRi/
4udQOeiwaIIdKEo8XUBXpBdEdlG3lzHYyOwCuxwzP/fyQY5F42VHAISX3I4KosIdJi
AxNdhZOogAskhIPElh6OyiECHCUh8HUaW3h00u8whIPFlzl5ppllGQOLLsg3VcjKHg
MSXOXulmWYZAYkvyzZUy8kcAqkWX+aQ0ExFIGACEl/AwDWcCDgCEp8jIS8CAROQ+AI
GruFEwBGQ+BwJ+VQTUP4YAhJfDBDdikBQBCS+oEhrHBGIISDxxQDRrQgERUDiC4q0x
hGBGAI5LL4YEroVgYAJSHwBA9dwIuAISHyOhLwIBExA4gsYuIYTAUdA4nMk5HOYQPc
sXeLrHu4aVQRCEp8OgQh0EwGJr5vAa1gRkPh0BkSgmwhIfN0Evt1h1ZgTBCS+nNhmL
TIdCUh86bgrmlNOEJD4cmKbtch0JCDxpeOuaE45QSAh8eUECS1SBAImIPEFDFzDiYA
jIPE5EvIiEDABiS9g4BpOBBwBic+RkE+IgIKSR0DiSx5LZRKBDhGQ+DqES8EikDwCE
l/yWCqTCHSIgMTXIVwKFoHkEch08SWPhDKJQMAEJL6AgWs4EXAEJD5HQl4EAiYg8QU
MXMOJgCMg8TkS8plOIOPmL/Fl3JZpwtlCQOLLlp3UOjKOgMSXcVumCWcLAYkvW3ZS6

8g4AhJfyrZMiUWgfQISX/t81CoCKSMg8aUMrRKLQPsEJL72+ahVBFJGQOJLGVolFgF
HIL6X+OJzUa0IpJyAxJdyxBpABOITkPjic1GtCKScgMSXcsQaQATiE5D44nPJ7lqtL
i0ISHxpsQ2aRC4SkPhycde15rQgIPGlxTZoErlIQOLLxV3XmtOCQFqILy1IaBIiEDA
BiS9g4BpOBBwBic+RkBeBgAlIfAED13Ai4AhIfI6EfFoQyKVJSHy5tNtaa1oRkPjSa
js0mVwiIPHl0m5rrWlFQOJLq+3QZHKJgMTX/m6rVQRSRkDiSxlaJRaB9glIfO3zUas
IpIyAxJcytEosAu0TkPja56NWEXAEku4lvqQjVUIRSIyAxJcYJ0WJQNIJSHxJR6qEI
pAYAYkvMU6KEoGkE5D4ko40qIQaJ9MJSHyZvoOaf8YSkPgydus08UwnIPFl+g5q/hl
LQOLL2K3TxDOdQPLEl+kkNH8RCJiAxBcwcA0nAo6AxOdIyItAwAQkvoCBazgRcAQkP
kdCPnkElCkhAhJfQpgUJALJJyDxJZ+pMopAQgQkvoQwKUgEkk9A4ks+U2UUgYQI5IT
4EiKhIBEImIDEFzBwDScCjoDE50jIi0DABCS+gIFrOBFwBCQ+R0I+Jwik0yIlvnTaD
c0lpwhIfDm13VpsOhGQ+NJpNzSXnCIg8eXUdmux6URA4uve3dDoOUxA4svhzdfSu5e
AxNe9/DV6DhOQ+HJ487X07iUg8XUvf42ewwRixJfDJLR0EQiYgMQXMHANJwKOgMTnS
MiLQMAEJL6AgWs4EXAEJD5HQj6GgG5TTUDiSzVh5ReBNghIfG2AUbUIpJqAxJdqwso
vAm0QkPjaAKNqEUg1gcwRX6pJKL8IBExA4gsYuIYTAUdA4nMk5EUgYAISX8DANZwIO
AISnyMhnzkEsmSmEl+WbKSWkXkEJL7M2zPNOEsISHxZspFaRuYRkPgyb8804ywh0K7
4fCUZDgAAEABJREFUqqqqhmNTs2StKVuGEotAZwh44mtpaclfvHhxwcqVKwsph2tra
wdWV1f/rLm5+UOSLsFC1OebN7OyxVtZJgIi0DkC4crKynNramrqdtppp3vz8/OfQHQ
vNDY2HkS6U/Py8qZh19F+G3Y1bX/kTTiL8l8GDRq0HBF64iVWlwiIQAcJhBsaGn5Pn
1oENx2/HrENQ4RLKddQfh2B5WOTS0tLr6Jur3A4bPUvU/5o9erVA/C6REAEOkEg3K9
fv/H027NHjx7fwNsbr4DPzXIEV47QjkSApdhLvPV+Qd1bWCntJcQORrCj8bpEQAQiB
Driwk1NTWHeakNKSkoewoZje3D//IYNGyiWXEl5Pwpnbtq06dqysrKTub8I/5/4ofh
XOzKYYkVABP5NIIyAXnC3vOGasGa7r6io2Gre2ZgxYz5zZXkREIGuE9AfTLrOUBlEo
FMEJL5OYVMnEeg6AYmv6wzTOYPmlsYEJL403hxNLbsJSHzZvb9aXRoTkPjSeHM0tew
mIPFl9/5qdWlMIGDxpTEJTU0EAiYg8QUMXMOJgCMg8TkS8iIQMAGJL2DgGk4EHAGJz
5GQD5iAhgvfeeedw2VioDMQ/BkIT5w48exMtgkTJpwjE4NEz0A6nfXwyJEjr8hkGzV
q1OUyMUj0DKTTWQ/n5eU1ysRAZyD4M6A/uLjf++VFIGACEl/AwDWcCDgCEp8jIS8CA
ROQ+AIGruFEwBGQ+BwJeRFwBALyEl9AoDWMCMQSkPhiieheBAIiIPEFBFrDiEAsAYk
vlojuRSAgAhJfQKC7Moz6ZicBiS8791WrygACEl8GbJKmmJ0EJL7s3FetKgMISHwZs
EmaYnYS6Iz4spOEViUCAROQ+AIGruFEwBGQ+BwJeREImIDEFzBwDScCjoDE50jId4a
A+nSBgMTXBXjqKgJdISDxdYWe+opAFwhIfF2Ap64i0BUCEl9X6KmvCHSBQJaJrwsk1
FUEAiYg8QUMXMOJgCMQmPhWrlxZaOYGNl9XV9fHfDIsmbmi55NI3kRionNGl2HSH9v
T6hYvXlywatWqkpaWlny7D8q6Mv+g5piN46RcfEuWLOlXXV09t0ePHtPz8/NPj4a4e
fPmy6LvXbmqquqb9LnW3cfztP9PdH1buaJjYvtEt7VVTiRvQ0PDdATUv60cVs/Y42t
qai60crSFw+GxcHnJ6gbxX3Nz85PEjbP7oCyRNSYyl7bWGNuXuFZ759pZ91W1tbWl7
j7bfZiDfgcwrsXmYhdgl2JWvi968cTNxI7E5qxZs2YPYl7CFlZWVo7DL8f+jF1A+4L
ly5fvTPkp69+7d+9f8iS/3crYj6l/Ax8i7nLqD7KnLnXrsWXAL7a2vLy8KzZt2vQL6
s7GbC5PMM4kyq8Qcxt+CnHHkuMOvJ+LmHG030P7OfiLaJ9FeSF2GfZDYo/FWznuGtn
4gbRvwN6kPIr+3hzpF6JuFfaq5bV7Z8QMp3w+Vkh5OjFLsTe5tz5vUp6LMAu5v5b1T
ifmm5g/r7KyslepX0N7qLS0dB3+T1i7F/1/St6/4731u2Duh1O/Dr+AN+j+lF/GFjL
nA/GLsY3Gm/bHqbsYfzI2lfGPsxyU/bzE+oyof5z71VgNXErxbu8nUI7eu+g1Xk7bC
9gFltsZ9/7e2Tkh9wLqHsbbPM5oamqazdxLXHznffr3DLP505jm19n4KdivKdvbaQP
eDgzui4un882ULkAYuwNoKGUDVEf9MMofYcdgh9O+uWfPnjewob/F8rnfnzGepc2uO

/mx1uBSN5P2nvX19fZ/1vI3yvdiE9mI8fg1Y8aM+QxfRv9P6DOaN8S5lC8sKSk5j3n
OpW4pOWzuIbyXa9iwYX+lz960nVxYWHgX8fY5Z2s5lD72EHgPfyPtcddIfYg+S7Gbe
QMd5/JaPbaBsb9C/uMp+xcxK7h5Bw5MMdyTviYImzPVoWrih/BmKaD+JiqeJv4Zyv6
8qLMx88y3ZcYEezpi5zBQITlmkstbv+vH10UPym9QP4n5n0XZrnrmcBbxP+OmOsK7D
3V9ydOT2DmU19MW4j46r8+IvgW0zyduPnlt/f7e0+bvHWV/jeR6mz67YL0xe0DaQ8o
eMP7ejRgxYj1xNbQXR+bxMuUri4uLrY5idl/eZyfQXmdjH8XOp2yb0Q//cfTSAbKWu
mLs/4qKiuzt9SHteWxIHbYFQTZzbwfpZu5PAOZjPF2vpu4uzF1nUygiF001t5FrdN+
+fb9F/FY2oYU263899RdHyiNp24FyT/x92GyekLO4t8ueuN7bmWReLvyJxCynsWDIk
CGb8GXYjpg73AOImUH+uGskLkT/fbGLiJlLrJ+Xtt25fxH/HOZf1E3m5gDsFsz6N+C
9tZDHHj59+vXrtwv53qH+aBjbkz92Xiuov5d2W/8CYm/ibWkCtSo7sIt4aBwVMXuA2
Tj1XuO2PzZaFTmMjYnRYh9kLldSP9weavgi7EzMRPEDYkcz/k/snjgvL3XRjKhuaaK
uCfFVEuf2fi0N0XsXvUY7K0uInWi/etB3GWV3eXsHO9vfcipXsK9T8YuwW1fxey8+6
y9PfDzRL+Ez77ts7q2Ur+FT8VQE8v3Y1dM+ivYrANlM+UAEdxZ+IUKbVF5e/hHlY2i
fhzfB2EGqp3yP5aF+Fk/d4Xjv9xm8vcF2w/+WmMn4G/D3EzsXb59fduiO7dWr10ncF
zHGM/Q/lM2fQYy1HU+9PZ1D9HW5HiLuDOoPthjqxzHHw7k3gdgbchx1N2BtrpHD9Ar
rO4j1v0Wcn5d8y6j/Brmuo+xfxNh696D+BMo34+/EDrEAOH6bOY8lVw22lvoR2FziW
s2LunOx71of2h7CJsDzfbuPZ7RfQby9QVo181BcSv1pVknMG9hX4XcyTF7Ef416O9w
hmOxL2zDst7TdRJ/dsJ9x7+el7DOibQpx0/Dn4x/Du71/nrK/dzFrvIO9Mn5Hm+CJ8
84ac/D3jjGWUf8V7FTyzsFupzyRPLnz5jMgBsi8mb01OGhNVm7POCDu8ypuGHBnRTd
UVFRsJa/3hoyujy7H9rG5uHbrz5ib3f32vI3VVrzlpb3VGocOHbqxsbHxdOZgb69W6
TmwJq5t6lsFxdzYGDbnmGp7KDW3Na/Y2K7c2/psDpYj4k+yso1NW7v7YHHWh7hWjKz
ejBzt7r3FICbvLWxl2bYE/KfRtk25V8NBa7bfQ+KtnMNmv9fGa0qsLg2ieKt4XxRpM
BVNAQISHxB0iUB3EJD4uoO6xhQBCEh8QNAlAt1BIPzMM8+cLBMDnYEAz0BEc+Hhw4f
3lYmBzkDwZyDMX/HulpWLQbkYBK0D/c7XHR/7GlMEICDxAUGXCHQHAYmvO6in2ZiaT
vcQkPi6h7tGFYGQxKdDIALdREDi6ybwGlYEJD6dARHoJgLpKL5uQqFhRSBYAhJfsLw
1mgj4BCQ+H4UKIhAsAYkvWN4aTQR8AhKfj0KFNCSQ1VOS+LJ6e7W4dCYg8aXz7mhuW
U1A4svq7dXi0pmAxJfOu6O5ZTUBia9D26tgEUgeAYkveSyVSQQ6REDi6xAuBYtA8gh
IfMljqUwi0CECEl+HcClYBByBrnuJr+sMlUEEOkVA4usUNnUSga4TkPi6zlAZRKBTB
CS+TmFTJxHoOgGJr+sM0yODZpFxBCS+jNsyTThbCEh82bKTWkfGEZD4Mm7LNOFsISD
xZctOah0ZRyBl4ss4EpqwCARMQOILGLiGEwFHQOJzJORFIGACEl/AwDWcCDgCEp8jI
Z8yAkocn4DEF5+LakUg5QQkvpQj1gAiEJ+AxBefi2pFIOUEJL6UI9YAIhCfQC6KLz4
J1YpAwATCVVVV/1tdXV2zcuXKQsoLAh6/Q8NVV1f/T2yHysrKSdTbNW/ZsmU7UPiQd
Ty7atWqQyg/is3j/mT8H7B1lJ+tra0d6PIQV0L9QuyllpaWMH4xtgI7H1uErcQuram
pedH1MU/dRnINxb+Ef5b231i9M+q+RZvfn/Ld2J+IOxF7ETuR+98RNwf/AnYXtpL7U
/CPMMdSlyvdPHO/Kt78mPe7tL3IGn5qc+beMYrL3mKIv5i4DyP7eCPl47EfY++Sx7h
a+5+4t32chY8un0nMe9iznN8jaLPrzffff38nCr+wehuDsrFfbmXqHuT+eexsu+9OC
+fl5X3GBHYIh8NTKe/I5G5nYk9jT1L+Jv5uAL3GIZ3qyh988MFetC3g/mHavmRl/NV
m0WXy+hf99yf+ZWwhoHe1OMoPs4mH4Tdgb1IehbeNsjgDO5x7E4yN9UOSHcv9ZXj/y
s/P78nNH7F3CwsLv4XvwTrWY+soj0VQD5eVlT1YWlp6EvdrKB9ZVFS0gbJ3NTU1XUT
sr/HH4JupvAF7jvhb6evK/015APXexRymUPg78d/HD8B/gh+B+RfjPEYfvz8N+zDXI
6mbhtm8flxSUnI63C2vzaeWmK3cn4/fkfmcyTi2D2Zfcbxg3CZv4m/BLsOeJt7fu7q
6uj7k9C/aX8JsH8YRt4CcV5tRtwp7lfJF1F+Of5H7u6wjfjV1tg/HM/8zmN9s9rTE2

```
pxRv4XypfCwffIZwTIue2JDMJiFXw0b28f/pHwJ/fvh++LXk/MDyoXNzc2/4H5SdJl
7i2mkbh3lrfgl2Cdbt26t4H4CtiP3oV69el2I/ycWos5y27lfwXrinvVEzzdMfN6W2
9mqVavinnV4fok+PmP77HyXBT7KpC6yzmz+MsrvUH6HA/QM/qu0LyguLp7jynvsscc
a4mq4LwbeG/ifE/MjytfElLn94gLeWV+UQvWA7uP6U/dXxluK3UzMcdwXb dq06Sjyf
bsH/3H/BvOYxAb+krr38DdSF+/aTHs/rJaDceWWLVuq8BMIPIAF/wHf1tWLcT9jTks
MWltBMfXnRu6PxTcz5lXk8Daa+7au/E8//dQOSAEBn9Mnnyf0YMoh+s7kwNxBuZH6t
7G9KdfB5B0zWCXEu76+/hL6/Zy1/Ahm/t4NGTJkE/XexRq/TMFEU0fcMMrRe7eBPfw
K4x/PuMdQPpT2MZhd/yCn7cMjtL9MxZWcCTsDFL+46DOQtunYC dREM+K23cvOYYh+i
7CHI5H/ZP+uxOZzbw/UC/B/w6LL1uc5+P0/6u2yB+BQWK4kj827DIEdFr1+GE0l51T
aZ8E17llP9HyTy+dtgztjPnHPOjxNKz5jW/ShQLMnlj017MnwCp2fZXLzmfh0Es7Dj
qB8Od4rc5gn0l7OvX2emWBuIMcV1N9NnSt7T0zuvYv2+yj0wEIs3kTi9QfUFHLti9k
baC7tH/Xu3fspwDxJ2a6N9iNiA3h6zIiUo91R3Hzt888/f4BxhtH3TuxQDtfd3Bdj1
bTHvRj317Tb5jXQ/18E/Rg7lrezHcwWyu4aAINnsZlU7MlD4EC8vbHsMPyK8bwnK3X
Rl9+fcVb37dt3PmP9joB/YN/hAfEU3pj/sqCg4FdW5in9Q2J2o7zC9sEMXjvR3+MF4
1Ly1UkAABAASURBVDZ5Dxo06EH6TSf+Hubp7x2C80ROW6ioqMgOwIeU8zDz0fu1O3z
t8/o52uytZ/N17D6lzl2LKNxKXhMxRf9aD5dzEekKanxGsW9e2ryLOR6G2VmzuRiHT
1j7eq8xFBoEU9vH87jfQs6peHsoNbsyTPoR/y2Lww/EXoPXWez7U5R/jh1Nn/Ngdil
+b/xlMLqe9gfo+3ZeXl7cs05cQuebXD5v8vsX48Y966zV1uAzDgPrROxC7CzsP3iav
cXiXo3Y7dR9HxvP/Uy8V8YvxL6CnYo9Sts4VD2b8llR5Vbf1LS/gX2Vw3XysGHDlhD
r9efQzQXEK0z4IBub+qMB+A1ib+agLOX+NLeqSG77lHNV9tkyj5g9sINHjx5djx9A3
JHYszyZjmO8b5HrSutAW8XixYsLgGAi8oz6BtoPYvyxe++99wpi9sOKmGMlOeZQtk+
WEL6ce8t7OeX96Gd1462MHUWOCWzarS43h/hE4v3+lKd+/PHHXyPuDuIruF+MGbdD8
fZGOZ36sfaUxhdgf6Te7cNi7j1e+DZ503Ycdg/2Zfr6e1dcXLzW5mvGOptpPxCBnsV
c/o84m4Ptne2XvQm+Qft12E+IOZqYb1s/7g8xb0Yfyz2RvK3efMSMt3YzyvtF/PjIm
irsPpoRotmJupPJZ3OwfTyXfvcz5hX4YuqNt507ry91FZjLa+Wfc+/25TH6fY99W0C
/MZRvwOyraSox/431x26kbgbrOoWYM4qLi+OedeISOt/E+bxZh38xRtyzzpj29egzt
jef3ynVBTa+yTYiepyhQ4dubGxsPJ0JN7j68vLyza7cFV9RUbE1djyrA4JtqmdsViX
zao4ev7Njshnnu9zkeyg2j40dW9dd9zC231NbDc+hPIF5J30fogeJZQSv6C+b6NCUl
NnrpnhrT/ZgNk7s2bMxohkHKj4bPNaYZPOIESPcp0Zss+4DJMCh/CjA4XJyqGjG3S6
+nNwBLVoEICDxASF9Ls0klwhIfLm021prWhEIz5kz5xiZGOgMBH8GwmPGjBkqE4POn
oFRo0YVpYN1dv7d2S88fPjwW2Ri0NkzMHLkyNnpYJ2df3f2a/93vrT6QtZkRCC7CEh
82bWfWk0GEZD4MmizNNXsIiDxZdd+ajUZREDiy6DN6tapavCkE5D4ko5UCUUgMQISX
2KcFCUCSScg8SUdqRKKQGIEJL7EOClKBJJOIGPFl3QSSigCAROQ+AIGruFEwBGQ+Bw
JeREImIDEFzBwDScCjoDE50jIZyyBTJ24xJepO6d5ZzwBiS/jt1ALyFQCEl+m7pzmn
fEEJL6M30ItIFMJSHzJ3zllFIGECEh8CWFSkAgkn4DEl3ymyigCCRGQ+BLCpCARSD4
BiS/5TJVRBByBdr3E1y4eNYpA6ghIfKljq8wi0C4Bia9dPGoUgdQRSIn4WlpawkuWL
OmXuml3PPPKlSv7Y3u6nnV1dX1cOda31xYbq3sR6CyBdsVXU1Mzo6qq6uSOJl+1atX
QPn36/G9H+yUa35l5hcPhsfn5+S+5MRoaGqYjxv7uPtpv3rz5MrtnnKtqa2tLrRxtM
Hmwurr6eexs7DTiXsOuj45Jz7JmlU4EwhyaizhAr3CgBuDnRU+usbFxVl5e3oXRdds
rk+NJ3nyPWxw5h3O/Dr8A+yljncj9HygPt3Zn1L2MLaT9Qvxzy5cv3xn/EPYcsQvwD
7tY852ZV1lZ2avMa431J6eNfz7lQsqPk381VmNC4/5y4g6yMv6Mpqam2TxMSoj1L5j
YWz2MX0HlOcS9hk2lrEsEEiYQ5tAcT/SHHKS78fdg/lVeXr6Z+qWVlZWT/EoKHNTx2
```

NNmCGaa+YjNpnkX+kzDh3rwH/4NDr71v6m5uflc7oto3zESbznOoa6QQ35YSUmJ9e/
FW7OQut7MrbB3795HUd4V869OzivEuHmWhPmYaN7hTWgCKqBuPmPNZ35jaZtJuWdRU
VE1/mXariwuLq7B+1d9ff1U5juVdns42dfDUBo3YLpEIGECdnCe41BeRY8jSktLH8G
3ujiQl/DJZu1+PXGLsKPMEMwd5iNmb8nPOJS3+sGh0EYrc6g3Mk5vbB6xfn/Kd9LeY
ILC29W4devW31NowUKDBw9uwHtlvH91Yl7WdwWiv5cHxmRuDsBuwZhuSxPzaiKnfQn
cRnk0MSfStgi7NfbNN2jQoOsR7gN0fJs+jxJTamW8LhFImECYw38dB2c6Zgd+m44R0
cyh3YS6TXtsBfm+zuE9GH88b4+l+NMshkM/Fj8IuwFrdRFziKugfDj9J+OPYexJlJs
p++0ujjYTc8Lzsn7kORf7Lg8MewDsQfkEbAq5puHPx8+h7TzKu+Ef4v52yhNj33y0z
eDNdwrtZ2A38SacgPfe9jaOTAQSIeAJijfbTA7P9LY6cNhmmwjaao+tJ97eVq2qN23
aVM2BPaBVW6uIf98kEmPRxHVoXtYnGQaLJt7Un7hcFRUVW11ZXgQSJeCJjyf72kQ7d
DZuzJgxn0Uf2M7mUT8RyBYCnviyZTFahwhkEgGJL5N2S3PNKgISX1ZtZwYvJgenHn7
xxRevlomBzkDwZyA8cuTIP8rEQGcg+DMQ3mOPPV6XiYHOQPBnQL/z5eDvGlpyehCQ+
NrYB1WLQKoJSHypJqz8ItAGAYmvDTCqFoFUE5D4Uk1Y+UWgDQISXxtgVC0CjkCqvMS
XKrLKKwLbISDxbQeQmkUgVQQkvlSRVV4R2A4BiW87gNQsAqkiIPGlimzq8ipzlhCQ+
LJkI7WMzCMg8WXenmnGWUJA4suSjdQyMo+AxJd5e6YZZwmBJIgvS0hoGSIQMAGJL2D
gGk4EHAGJz5GQF4GACUh8AQPXcCLgCEh8joR8EggoRUcISHwdoaVYEUgiAYkviTCVS
gQ6QkDi6wgtxYpAEglIfEmEqVQi0BEC2S2+jpBQrAgETEDiCxi4hhMBRyDp4lu5cuU
uy5cv39kNUFdX18eVY317bbGxuheBbCPQrvhqampmVFVVndyRRefn508uKCh42vVpa
GiYjiD7u/tov3nz5svsnnGuqq2tLbVytDH2g9XV1c9jZ2OnEfcadn10jMoikKkEwhz
mizjYr3DQB+DnRS+ksbFxVl5e3oXRddsrl5aW3t/S0rLJ4sg5HH8+Vkj5cfKvxmpMa
NxfTtxBVsaf0dTUNHvVqlUlxPoXY/fjJoxfgT+HuNewqZR1iUDHCKRhdJjDfDzz+pA
Dfjf+Hsy/ysvLN1O/tLKycpJfSQEBjceeNkO808xHbDbNIfrkmS8rKzPRvMPb0ARUQ
N18xpvf3Nw8lraZlHsWFRVV41+m7cri4uIavH/V19dPRZRTabeHgL2lh9K4AdMlAhl
PwA70c4jlKlZyBG+tR/CtLoRySTgctna/nrhF2FFmJSUld5iPmP+WRIw/RJiT6XQAd
guGhlqaGKuJnPbGvY3yaGJOpG0Rdmvsm2/QoEHXI9wH6Pg2fR4lptTKeF0ikPEEwoj
mOg70dOz38VbDG2ojIplDuwk1Xsg2deQ8BPslwpyH3wM7AZtCrmn48/FzaDuP8m74h
7i/nfLE2DcfbTN4851C+xnYTbwJJ+CnbTOgKkQgAwl4guLNNpNDPb2t+SOC2Qiwua3
2VNUzZhOfvp+4/BUVFVtdWV4EMp2AJz7eOGszfSGZNn/NVwQ88QmDCIhA8AQkvuCZa
0QR8AiE77vvvl1lYhB9Bm6//fZdZalnEB49evRlMjGIPgPjx4+/TJZ6BuHBgwf/YvD
gwTJxyNozwF/Mb0hHCw8ZMmS9TAyy+Qz079//43Q0/cHF+9VXP0QgeAISX/DMNaIIe
AQkPg+DfkQRUDEgAhJfQKA1jAjEEpD4YonoXgQCIiDxBQRaw4hALAGJL5aI7kUgIAI
ZIL6ASGgYEQiYgMQXMHANJwKOgMTnSMiLQMAEJL6AgWs4EXAEJD5HQj4DCGTXFCW+7
NpPrSaDCEh8GbRZmmp2EZD4sms/tZoMIiDxZdBmaarZRUDi68p+qq8IdIGAxNcFeOo
qAl0hIPF1hZ76ikAXCEh8XYCnriLQFQISX1foqa8IOAKd8BJfJ6Cpiwgkg4DElwyKy
iECnSAg8XUCmrqIQDIISHzJoKgcItAJAhJfJ6BlQhfNMf0JtCu+mpqaGVVVVSen/zI
0QxHIPAJhBHZRdXX1K4hsAH5e9BIaGxtn5eXlXRhdt70yeX5Knr/j78Bfis3F7qusr
ByDf4vxbqurq+tD++OUL8bfTv1arAa7jLrf4OfiZ2xvLLWLQCYTCLe0tBzPAj5EZHf
j78H8q7y8fDP1SxHOJL+SAuIYjz1thkimmY/Y7HA4XEifmWVlZdMIPR3bgBU2NTV9i
G9obm7evb6+PkxMH8bua/H4J7H52DhsMnHWZwBelwhkLQH77HwOIVzFCo8oLS19BN/
qQiyXIBBr9+uJW4QdZVZSUnKH+Yh5b0kEVG/B5J1DuR/+44KCglLqaikPGzNmzGeUi
7AzsRB1TRHL4/5VrCe2DtMlAllLIIxorkMg07Hfx1slb7CNCMNEZEKNF9KqDjFeQc6
5Vkn5mt69e59aXFz8feoWMca5+LHWxptwX9qHYd+j7vyInYQ/gbZzqL/Z4mQikK0EP

```
EHxZrPPxOltLRIhzEaAzW21t1c/ZMiQTfRtshgTsnmzyCdt3Jy0fWIxMhHIZgKe+Iq
Li9dm8yK1NhFIRwKe+NJxYpqTCGQ7AYkv23c4/danGUUISHwREHIiEDSB8COPPPI1m
RjEnoEHH3zwa7LUMgjvs88+E2RiEHsG9t9//wmy1DIIjxw58jqZGOgMBH8G9DtfKOg
vfY0nAl8QkPi+4KCfIhA4AYkvcOQaUAS+ICDxfcFBP0UgcAISX+DINWD6E gh2ZhJfs
Lw1mgj4BCQ+H4UKIhAsAYkvWN4aTQR8AhKfj0IFEQiWgMQXLO+OjaborCYg8WX19mp
x6UxA4kvn3dHcspqAxJfV26vFpTMBiS+dd0dzy2oCHRJfVpPQ4kQgYAISX8DANZwIO
AISnyMhLwIBE5D4Agau4UTAEZD4HAn5DhFQcNcJSHxdZ6gMItApAhJfp7Cpkwh0nYD
E13WGyiACnSIg8XUKmzqJQNcJZIv4uk5CGUQgYAISX8DANZwIOAISnyMhLwIBE5D4A
gau4UTAEZD4HAn5bCGQMesI19TUXFRdXf1KVVXVAPy8jJm5JioCGU4g3NLScjxr+DA
vL+9u/D2YLhEQgQAI2GfncwjvKsY6orS09BG8LhEQgQAIhBHcdbz9pmO/D2A8DSECI
hAhYG++UDgcnllWVjY9UieXVAJKJgLxCXjiKy4uXhu/WbUiIAKpIuCJL1XJlVcERKB
tAhJf22zUIgIpJSDxpRSvkotANIHW5fALL7xwiUwMdAaCPwPhkpKSZTIx0BkI/gyE+
SeGZ2VlYlAmBkHrQL/ztf4M150IBEZA4gsMdRoOpCl1KwGJr1vxa/BcJiDx5fLua+2
KDlMTAAALXklEQVTdSkDi61b8GjyXCUh8ubz7Wnu3Ekgr8XUrCQ0uAgETkPgCBq7hR
MARkPgcCXkRCJiAxBcwcA0nAo6AxOdIyKcVgVyYjMSXC7usNaYlAYkvLbdFk8oFAhJ
fLuyy1piWBCS+tNwWTSoXCEh8ie2yokQg6QQkvqQjVUIRSIyAxJcYJ0WJQNIJSHxJR
6qEIpAYAYkvMU6KEgFHIGle4ksaSiUSgY4RkPg6xkvRIpA0AhJf0lAqkQh0jIDE1zF
eihaBpBGQ+JKGsrsSadxMJeCLb+XKlf0zYRHvv//+TvHmWVdX18fqq6qqhi5ZsqSfl
eNZvHW2tLTkL168uCBefEfrLA9jFHa0n8Vb31WrVpXYfOy+q2Z5LGdn8zimifZn3bs
sX75850Tjcz3OF19+fv6F3QWjpqbmqtra2tJ445uQqqur51ob/vnNmzefbeVYo/4yq
wuHwz/t06fPz60cz2h/IraesccPHDjw61bf3lysfXu24447ju/Ro8ch24uL1z6I/5q
bm59kDuPitbdVxwPnm7C5Nra9trbWX1dsW7x7coxnbP8cNDQ0TEdQCT+UOUOTCwoKn
o6XW3XbEvDEB/Dv0HQCFmIjL2cTXsAuoFyBfwN7ifJ0/FLsTYuLNtp+Sv3f8XeQ6ze
U5+Jn4J/HXsfup83PZX2pexlbSNw1PKHPaGpqmm1PfeIeN6NtosX17t37l7TfTt0p3
O+LeAorKyvH0T4X+xt13pyJOcjK2G8x7yJ39Fx+RvxLeXl5e3mNUT8Y+zrylnNYS8n
jz8XisYWR8T6kbPYoc/EZRaUJMV4v8txA3QgsRPyb2FwOcCH+D5it+Uz8cuzP2AUW5
6y0tHQd5T9htiafl729iH0bq2WMYxm/FSPWdMWmTZt+QbvFrKP9r5bDrQuuh9C2GPv
Y6vE2D2N/IbGzuF+I2cPrWtY/nbpvYsOJPR8rpOzPxd6GxK7HljGXYtr9i/nfT/9NV
kGfy2l/kbi7MI89/l1rk31BwBNfSUnJb4G23qo4PG/jd8F6s6knYddTbsJ2pHwH/hO
s1UWfQtpmlpWVTSPPZBo3YAOwnth/YmW0+7k45AOpK+RwHMbYV9PnZe6vLC4uriHX2
8R6b0Hq8ynvT177PzJ5gPv3iL+GmCHE98D2xEK0z6TNxrJb36iLnsskDoe9kT71AyI
FxvgNb5xeRUVF1fTx5kLT7lgJVscTfRj19tAxNiYw8x4j2k0odlBDzK2BXHcR6+Zi+
YbwVrZP2iIEchRt36bPR9gx2OFY3Is8Pi/eykcQ9BfsKayQ9fuMONDjyblmzJgxn+H
raH+Lvt7/zTfeWxdrOwn7EW21mF2O/WxijKHt16GUb6LxaXg+g62g/A5rD1Pvz4U8g
7j/G2Pdi03kwbJL9K8CtOXRL4Q/Bh6HUh6DHRphv5WyrggBT3yRsnPFgFvCzUTgPoF
dQbkfG76JcgPlFmybi7Z6rzIUehVvh8+e4pb/Udo+xPxcHHLb7Iby8vLNxNq1iB+38
oQuYXNNmPXM4T94cl5N/V1Yq4s2E2dfKnuRN0zcbdSNxp/Y2Nj4FvWH8OQ9Eu/PhbH
FHNQ/Urcj5l/EDeXmPPqfxENhFGVvLqz3H5Q/xPLoW0e7PYAaubfLZ2SfxbQts8pYo
589PPr069fPhPoRb/GnyPsk9Vt48DTHxts9uRZgNxH3GnEe+w0bNsyn7QDsW/BpxHx
GxF6PXUybHXh/jtHrov3P5JtJzDDMrmj2ZVQYE8Ly3qF8NJymwNIeXDbmLczD37t/8
h/3W8nnnQP8Q1u2bJlFP/+i/w+5sbfefJJWYy8znwXU7YDpihAwcdjn0ozIvT2972B
zz+NJdTRPv1c5JF+hfCBPsZvxd2L29nDhnqftCuq938vwJ9DnHOpuprEJ8IeTZyrm5
```

```
6I+RJyfh7bbuZ9obz78pbwhplh/+tZzf4/FmxE3yby19erVazJtfYlp5t7muxv+IQT
9CfX7EPss3p8L9xcTO6W+vn4EB+FZZxyknsQdhO3HQ2Epcd5chg4dWkXdgazlLPxCb
Ap2lBnj+IzsjUOdx5FDN5Z80ziQi22eiO3bjDc2sq6j4foN+t7MGJOY50f0s7efhfp
G+0PYBPo8ztge+4qKiq3M3Q6vPQxeoJ/PiI5zubcHnTH158gYq6n31kXMS9gbzM2bF
/U+e8YaxziHUzeZMdfiR2BzqZ+H3wM7gVz+3kXWO5n2G2i7nzb7ojiN/N5F3SHYL7G
fkPdo4r7NuK8z//ewhV6QfngEvEND6SFgHYb3LoBu9Ar84JC4txN3iV308T5NgX8C8
O1t6XWkfru52KAm22DrQN9WT1SrczZkyBDvdwt335ZnTG8u1k6+BjvIrO9IZ8OGDau
0trYsun90DP19Rq6eh0b11q1bp3CI7bCHbI42nmsn13bX72LNR8cj3Fk8cA62cRNlZ
DnMWPcqBHALfb2Hl9U5I1dz9DiuPtYnEtNWH87B8zyQfsU8vhcbk8v3nviAsso2Idk
g2DD73SbZadM2nz00RowY4f3unOxJIpzVJubO5qW//Q7X2e5d6sc52MwDqaZLSbKws
ye+LFyXlpQOBDSHdglIfO3iUaMIpI6AxJc6tsosAu0SkPjaxaNGEUgdAYkvdWyVWQT
aJZBT4muXhBpFIGACEl/AwDWcCDgCEp8jIS8CAROQ+AIGruFEwBGQ+BwJ+ZwikA6Ll
fjSYRc0h5wkIPHl5LZr0elAQOJLh13QHHKSgMSXk9uuRacDAYkvHXYhFNIscpCAxJe
Dm64lpwcBiS899kGzyEECEl8ObrqWnB4EJL702AfNIgcJtCG+HCShJYtAwAQkvoCBa
zgRcAQkPkdCXgQCJiDxBQxcw4mAIyDxORLybRBQdaoISHypIqu8IrAdAhLfdgCpWQR
SRUDiSxVZ5RWB7RCQ+LYDSM0ikCoCmSe+VJFQXhEImIDEFzBwDScCjoDE50jIi0DAB
CS+gIFrOBFwBCQ+R0I+8whk+IwlvgzfQE0/cwlIfJm7d5p5hhOQ+DJ8AzX9zCUg8WX
u3mnmGU5A4kviBiqVCHSEgMTXEVqKFYEkEpD4kghTqUSgIwQkvo7QUqwIJJGAxJdEm
EolAo5AIl7iS4SSYkQgBQQkvhRAVUoRSISAxJcIJcWIQAoISHwpgKqUIpAIAYkvEUq
ZH6MVpCEBiS8NN0VTyg0CEl9u7LNWmYYEJL403BRNKTcISHy5sc9aZRoS6CbxpSEJT
UkEAiYg8QUMXMOJgCMg8TkS8iIQMAGJL2DgGk4EHAGJz5GQ7yYCuTusxJe7e6+VdzM
Bia+bN0DD5y4BiS93914r72YCEl83b4CGz10CEl/s3uteBAIiIPEFBFrDiEAsAYkvl
ojuRSAgAhJfQKA1jAjEEpD4YonoXgQcgRR7iS/FgJVeBNoiIPG1RUb1IpBiAhJfigE
rvQi0RUDia4uM6kUgxQQkvhQDTmZ65couAhJfdu2nVpNBBCS+DNosTTW7CEh82bWfW
k0GEZD4MmizNNXsItAV8WUXCa1GBAImIPEFDFzDiYAjIPE5EvIiEDABiS9g4BpOBBw
Bic+RkO8KAfXtBAGJrxPQ1EUEkkFA4ksGReUQgU4QkPg6AU1dRCAZBCS+ZFBUDhHoB
IEsFV8nSKiLCARMQOILGLiGEwFHQOJzJORFIGACEl/AwDWcCDgCEp8jIZ+lBNJ3WRJ
f+u6NZpblBCS+LN9gLS99CUh86bs3mlmWE5D4snyDtbz0JSDxBb03Gk8EIgQkvggIO
REImoDEFzRxjScCEQISXwSEnAgETUDiC5q4xhOBCIFwxMuJgAgETEBvvoCBazgRcAQ
kPkdCXgQCJiDxBQxcw4mAIyDxORLyISEIloDEFyxvjSYCPgGJz0ehgggES+D/AwAA/
/+2fO0YAAAABklEQVQDAIeoF3EzBiwIAAAAAElFTkSuQmCC)
3
4  ---
5
6  # Complete Migration Script
7
8  ```Python
9  #!/usr/bin/env python3
10
11  import argparse
12  import asyncio
13  import aiohttp
```

```
14  import json
15  import logging
16  import yaml
17
18  from pathlib import Path
19  from config import *
20
21  CHECKPOINT_FILE = Path(
22      "checkpoints/migrated_issues.json"
23  )
24
25  logging.basicConfig(
26      filename="logs/migration.log",
27      level=logging.INFO,
28      format="%(asctime)s %(levelname)s %(message)s",
29  )
30
31  class GitLabMigration:
32
33      def __init__(
34          self,
35          mappings,
36          dry_run=False,
37          limit=None
38      ):
39          self.mappings = mappings
40          self.dry_run = dry_run
41          self.limit = limit
42
43          self.source_project = \
44              mappings["source"]["project_id"]
45
46          self.dest_project = \
47              mappings["destination"]["project_id"]
48
49          self.checkpoint = \
50              self.load_checkpoint()
51
52          self.dest_users = {}
53
```

```
    def load_checkpoint(self):

        if CHECKPOINT_FILE.exists():
            return json.loads(
                CHECKPOINT_FILE.read_text()
            )

        return {}

    def save_checkpoint(self):

        CHECKPOINT_FILE.parent.mkdir(
            parents=True,
            exist_ok=True
        )

        CHECKPOINT_FILE.write_text(
            json.dumps(
                self.checkpoint,
                indent=2
            )
        )

    async def gitlab_get(
        self,
        session,
        url,
        token
    ):
        headers = {
            "PRIVATE-TOKEN": token
        }

        async with session.get(
            url,
            headers=headers
        ) as r:
            r.raise_for_status()
            return await r.json()

```

```
94      async def gitlab_post(
95          self,
96          session,
97          url,
98          token,
99          payload
100     ):
101
102         headers = {
103             "PRIVATE-TOKEN": token
104         }
105
106         async with session.post(
107             url,
108             headers=headers,
109             json=payload
110         ) as r:
111             r.raise_for_status()
112             return await r.json()
113
114     async def fetch_issues(
115         self,
116         session
117     ):
118
119         url = (
120             f"{SOURCE_GITLAB_URL}"
121             f"/api/v4/projects/"
122             f"{self.source_project}"
123             f"/issues?per_page=100"
124         )
125
126         issues = await self.gitlab_get(
127             session,
128             url,
129             SOURCE_TOKEN
130         )
131
132         if self.limit:
133             issues = issues[:self.limit]
```

```
134
135         return issues
136
137     async def resolve_user(
138         self,
139         session,
140         username
141     ):
142
143         mapping = self.mappings["users"].get(
144             username
145         )
146
147         if not mapping:
148             return None
149
150         dest_username = mapping[
151             "dest_username"
152         ]
153
154         if dest_username in self.dest_users:
155             return self.dest_users[
156                 dest_username
157             ]
158
159         url = (
160             f"{DEST_GITLAB_URL}"
161             f"/api/v4/users"
162             f"?username={dest_username}"
163         )
164
165         users = await self.gitlab_get(
166             session,
167             url,
168             DEST_TOKEN
169         )
170
171         if users:
172             self.dest_users[
173                 dest_username
```

```
            ] = users[0]["id"]

            return users[0]["id"]

        return None

    async def migrate_issue(
        self,
        session,
        issue
    ):

        iid = str(issue["iid"])

        if iid in self.checkpoint:
            return

        title = issue["title"]

        if self.dry_run:

            print(
                f"[DRY RUN] "
                f"Would migrate issue "
                f"#{iid}: {title}"
            )

            return

        assignee_id = None

        if issue.get("assignee"):
            assignee_id = \
                await self.resolve_user(
                    session,
                    issue["assignee"][
                        "username"
                    ]
                )

```

```
            payload = {
                "title": title,
                "description":
                    issue.get(
                        "description",
                        ""
                    ),
                "labels":
                    ",".join(
                        issue.get(
                            "labels",
                            []
                        )
                    )
            }

            if assignee_id:
                payload[
                    "assignee_ids"
                ] = [assignee_id]

            create_url = (
                f"{DEST_GITLAB_URL}"
                f"/api/v4/projects/"
                f"{self.dest_project}"
                f"/issues"
            )

            created = await self.gitlab_post(
                session,
                create_url,
                DEST_TOKEN,
                payload
            )

            await self.migrate_notes(
                session,
                issue["iid"],
                created["iid"]
            )
```

```
254
255             self.checkpoint[iid] = True
256             self.save_checkpoint()
257
258         async def migrate_notes(
259             self,
260             session,
261             source_iid,
262             dest_iid
263         ):
264
265             notes_url = (
266                 f"{SOURCE_GITLAB_URL}"
267                 f"/api/v4/projects/"
268                 f"{self.source_project}"
269                 f"/issues/"
270                 f"{source_iid}/notes"
271             )
272
273             notes = await self.gitlab_get(
274                 session,
275                 notes_url,
276                 SOURCE_TOKEN
277             )
278
279             for note in notes:
280
281                 payload = {
282                     "body": note["body"]
283                 }
284
285                 destination_url = (
286                     f"{DEST_GITLAB_URL}"
287                     f"/api/v4/projects/"
288                     f"{self.dest_project}"
289                     f"/issues/"
290                     f"{dest_iid}"
291                     f"/notes"
292                 )
293
```

```
294                 await self.gitlab_post(
295                     session,
296                     destination_url,
297                     DEST_TOKEN,
298                     payload
299                 )
300 
301         async def run(self):
302 
303             timeout = aiohttp.ClientTimeout(
304                 total=REQUEST_TIMEOUT
305             )
306 
307             semaphore = asyncio.Semaphore(
308                 MAX_CONCURRENT_REQUESTS
309             )
310 
311             async with aiohttp.ClientSession(
312                 timeout=timeout
313             ) as session:
314 
315                 issues = await self.fetch_issues(
316                     session
317                 )
318 
319                 async def worker(issue):
320 
321                     async with semaphore:
322                         await self.migrate_issue(
323                             session,
324                             issue
325                         )
326 
327                 await asyncio.gather(
328                     *[
329                         worker(issue)
330                         for issue in issues
331                     ]
332                 )
333 
```

```
def main():

    parser = argparse.ArgumentParser()

    parser.add_argument(
        "--dry-run",
        action="store_true"
    )

    parser.add_argument(
        "--limit",
        type=int
    )

    args = parser.parse_args()

    with open("mappings.yml") as f:
        mappings = yaml.safe_load(f)

    migration = GitLabMigration(
        mappings,
        dry_run=args.dry_run,
        limit=args.limit
    )

    asyncio.run(
        migration.run()
    )

if __name__ == "__main__":
    main()
```

Requirements

```
aiohttp>=3.10.0
PyYAML>=6.0
```

Runbook

1. Create Migration Project

Confirm destination project:

```
83486174
scicomp-kanban_cce-migration
```

2. Generate Tokens

Source

PAT scopes:

```
api
read_api
read_repository
```

Destination

PAT scopes:

```
api
```

3. Configure Environment

```
export SOURCE_GITLAB_TOKEN=<token>

export DEST_GITLAB_TOKEN=<token>

export PAT_JHERRERA15=<token>

export PAT_JBANFELDER=<token>
```

4. Validate User Mapping

Source	Destination
jherrera	jherrera15
jbanfelder	jbanfelder
source_username_2	dest2

5. Dry Run First 50 Issues

```
python gitlab_migration.py \
  --dry-run \
  --limit 50
```

Review:

- assignee mappings
- labels
- titles
- issue counts
- comments

---

6. Pilot Migration (50 Issues)

```
python gitlab_migration.py \
  --limit 50
```

Validate:

- 50 created
- comments copied
- labels copied
- assignments applied

---

7. Validation Checks

Source:

```
curl ...
```

Compare:

- total issues
- open issues
- closed issues
- labels
- milestones

---

8. Full Migration

```
python gitlab_migration.py
```

9. Resume if Interrupted

```
python gitlab_migration.py
```

The checkpoint file automatically skips migrated issues.

---

10. Final Validation

Verify:

- Issue counts match
- Open/closed counts match
- Assignees mapped correctly
- No failed migrations listed in:

```
reports/failed_items.json
```

For a production migration, I would additionally recommend adding:

- pagination support beyond 100 issues,
- automatic destination-user ID caching,
- milestone migration,
- attachment upload migration,
- retry/backoff handling for GitLab rate limits,
- CSV and JSON validation reports,
- verification mode that compares source vs destination after completion.